

COMPUTER GRAPHICS AND ARRAY PROCESSORS

Ian J. Curington
Computer Graphics Unit Manager
Floating Point Systems
Bracknell, UK

The image rendering phase of generating 3-D animated sequences needs extremely powerful computer systems to be productive in a short production schedule. Advanced architectures such as add-on array processors and highly parallel systems using the INMOS Transputer can provide this level of power for the animator. The paper shows some of the architecture that gives best advantage to RAY TRACING rendering algorithms and some examples of current experimental programs.



Ian Curington is the Computer Graphics Unit Manager and system specialist for Floating Point Systems where his activities include performance analysis, graphics algorithms, image processing, and scene simulation software. His interests include computational geometry, image synthesis, and motion dynamics for animation. He is a member of the ACM-SIGGRAPH, and EUROGRAPHICS.

ABSTRACT

The use of parallel processing in computer graphics has the advantage of allowing very high performance with existing integrated circuit technology. Applied to complex 3-D scene generation, parallel processing techniques dramatically reduce the rendering time for animated sequences. The latest parallel technologies, such as the INMOS Transputer, allow efficient implementation of Ray-Tracing techniques and other rendering algorithms to run in real-time. Parallel techniques are explained and examples shown of images generated on array processors and massively parallel supercomputers.

RENDERING

Production animation systems have many diverse components, usually organised by the type of tool required at each stage of the production cycle. After the story board and artistic design comes model building and motion dynamics. This can be previewed on fast graphics workstations, or by generating line tests on film or video. After most of the scene has been designed, the computer system is finally set to work generating high resolution full colour images for each frame in the animated sequence. The generation of these images is called the 'rendering' step. This single task can require 95% of the entire computer system, and may be the limiting factor for turning out the film segment within the production deadlines [16]. Array processors and highly parallel computer architectures provide the dramatic boost in performance just where it is needed: during the final rendering!

ARRAY PROCESSOR ARCHITECTURE

The characteristics of the FPS-5000 are similar to other array processors, but with the added feature of multi-processor parallelism. The FPS-5000 is a tightly coupled system, with up to 4 concurrent processors. Each processing element in the FPS-5000 contains a high speed pipelined CPU, operating at a 6MHz instruction rate, where each instruction is composed of 70 micro-instructions per clock cycle. In addition each processor is rated at a maximum theoretical performance of 18 million floating point operations per second (MFLOPS). All of the processors access a high bandwidth shared memory. Each processing element supports a vector math library, so that application vectorisation techniques are similar to vector supercomputers.

Within any given parallel computer architecture, a certain level of granularity in an application code will achieve the best performance. Granularity is a measure of the relative number and size of the parallel segments in a program. One large program segmented into two parts would have a large granularity, while one composed of 4096 parts would have a fine granularity. The high performance and functionality of each processor in the FPS-5000 indicates a large segment granularity is most efficient.

T-SERIES SUPERCOMPUTER

The FPS T-Series is a supercomputer-class computer with a highly parallel architecture. The performance ranges from 128 MFLOPS in the smallest configuration to 262 GFLOPS (billion floating point operations per second) in the largest configurations by using a large

number of homogeneous processing elements. Each processing element is composed of an INMOS Transputer, a high-speed dual-ported memory, and pipelined vector processing elements so that each node is therefore a complete processing system. Disk systems are distributed among groups of nodes. The software for the T-Series is based on OCCAM, with concurrent calls to vector unit high speed arithmetic. The Transputer is very efficient at handling graphics data structures with arrays passed to the vector unit for 16 MFLOPS per node vector performance. The system can mix both 32 and 64 bit floating point arithmetic, which may be tailored to the graphics algorithm requirements. The power of the T-Series makes it an ideal rendering engine, for executing scan-line, z-buffer, or ray-tracing types of hidden surface and shading algorithms.

RAY-TRACING

Ray Tracing is, perhaps, the simplest and most accurate method of synthesizing a complex three dimensional image. In physical terms, the algorithm retraces the paths of the light rays that enter the eye, taking into account the rays' reflections and refractions, and the effect of light scattering surfaces [17,18,22]. Unfortunately, ray tracing is extremely computationally intensive. The number of calculations can become large very quickly with increases in the number of objects in the scene, the depth of the ray tree, and the image resolution. This is further compounded by the more complex models that account for moving objects, shadowed objects and ray diffraction [3,12,13].

Ray tracing is so computationally intensive that massively parallel machines like the T-Series are an absolute requirement to make significant cuts in the time required to generate extremely complex images. New algorithms will also help improve computational efficiency. For example, 'space-tracing' methods [10] are an example of an approach which reduces the number of object-ray intersection tests and is nearly always faster than traditional methods. This and other methods can be applied to massively parallel systems. Ray tracing is easily segmented across many processors, so that each processor is computing different geometric elements in parallel [20,25]. In a fully parallel implementation, increasing the number of processors has the proportional effect of reducing the execution time.

In ray tracing algorithms, at least 85% of the execution time is spent in computing ray and object intersections. The type of intersection calculation depends on the complexity of the object description, but is usually some type of polynomial root finder. For quadric surfaces or polygonal objects, the ray and object intersection test can be vectorised using the standard math library forms. Special routines are used for the iterative root finding methods used with higher-order surfaces.

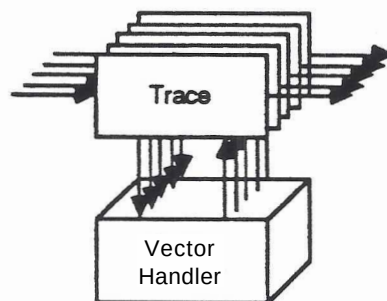
PARALLEL/VECTOR RAY TRACING

When arithmetic operations are performed on vectors in the T-Series floating-point arithmetic runs approximately twelve times as fast as for scalars. Techniques for vectorised ray tracing have been

discussed [14,26] where the rays are queued and sorted and vectors of rays are then tested against single objects. This speedup is available to both serial and parallel algorithms.

Vectorisation is possible when the ray tracing process is modified to queue rays until some arbitrary number are waiting to be tested. Once the queue is full each object in the scene database is tested against the list of rays. When all object-ray testing is complete the results are returned to the ray-tracing function.

The rays are distributed across multiple processing elements using the link communication of the INMOS Transputer. The HYPER-CUBE connectivity, used in the T-Series, allows fast transfer of a ray to other processors in any dimension of the network. Most of the rays for any small part of the scene are likely to be evaluated by concurrent processes on one node with a small number going out to other processors [27]. The usage of the vector processor on each node processing a ray queue is shown in the following figure.



An array of processes which use a common resource

SCAN-LINE RENDERING

After the geometric form of the scene has been determined, each pixel in the image must be assigned a colour appropriate to the light contribution to that point in the scene. Lighting models are used to determine the colour and intensity from the geometric information. One of the popular lighting models, and the most easily implemented, is PHONG shading. The PHONG shading model combines ambient, diffuse, and specular lighting components to derive the final shade at a surface point due to one light source. This vectorises well using the vector math library routines VSMUL, VSMA, VCLIP, VSQ, VSMSA and VMA [4,5,6].

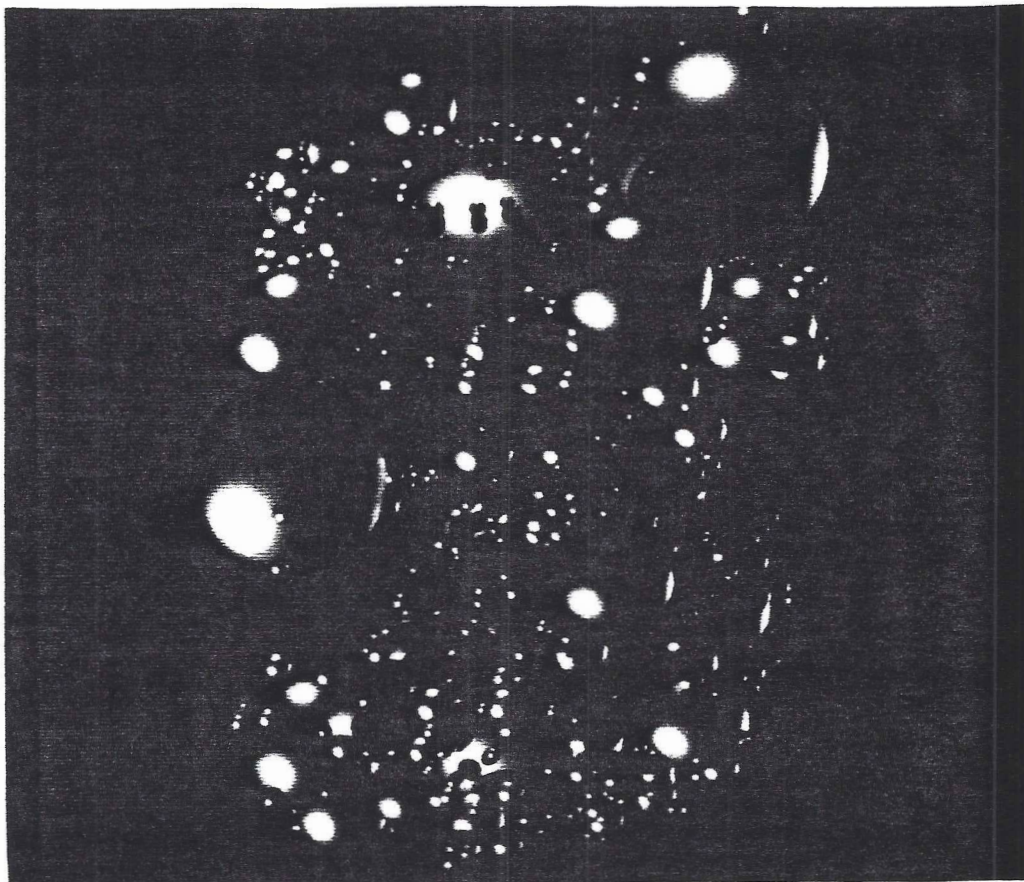
EXAMPLES

The photographs show examples of parallel and pipeline algorithms running on the FPS-5000 array processor. The first photo shows full multiple light source shading with shadows using a ray tracing algorithm, while the second shows the generation of synthetic ground terrain.

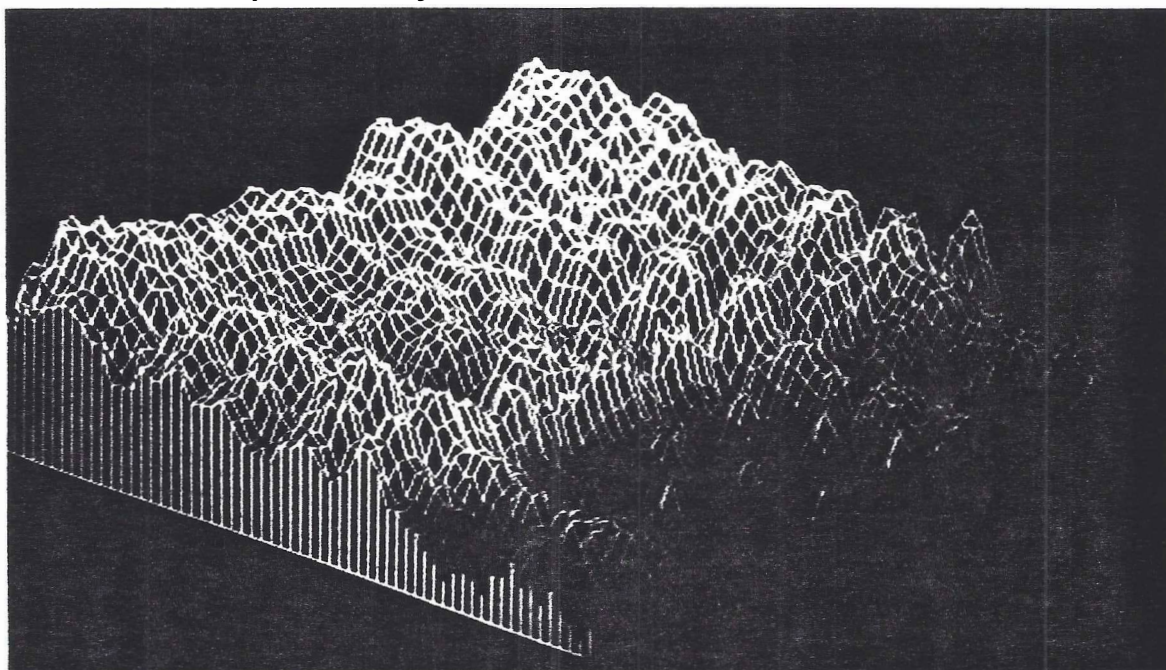
BIBLIOGRAPHY

- [1] Atherton, Peter "A Scan-line Hidden Surface Removal Procedure for Constructive Solid Geometry" Computer Graphics (SIGGRAPH Proc.), Vol 17, No. 3, July 1983.
- [2] Bronsvoort, Willem F., Jarke J. van Wijk, and Frederik Jansen "Two Methods for Improving the Efficiency of Ray Casting in Solid Modelling" Computer Aided Design, Vol 16, No. 1, Butterworth and Co. (Pubs) Ltd., January 1984.
- [3] Cook, Porter and Carpenter "Distributed Ray Tracing" Computer Graphics (SIGGRAPH Proc.), Vol 18, No. 3, July 1984.
- [4] Curington, Ian J. "Fast Vectorized Surface Shading on an FPS-5000" Proceedings of the 1985 ARRAY conference, New Orleans.
- [5] Curington, Ian J. "A Normal-Buffer Vectorized Surface Shading Model" EUROGRAPHICS '85 Proceedings Nice, France September, 1985, North-Holland Publisher.
- [6] Curington, Ian J. "Graphics in the Fast Lane: Notes on Array Processor Implementations of Graphics Algorithms" Application Note of Floating Point Systems, UK Ltd. 1985.
- [7] Curington, Ian J., "Parallel Subdivision for Synthetic Terrain Modelling" Proceedings of the FUSE Conference, Paris, May 1986.
- [8] Daniels, Paul A. and Curington, Ian J. "Particle Accretion Simulation with a Shaded Surface Display" Proceedings of the FUSE Conference, Munich, May 1985.
- [9] Dippe, M. and Swensen, J. "An Adaptive Subdivision Algorithm and Parallel Architecture for Realistic Image Synthesis" Computer Graphics (SIGGRAPH Proc.), Vol 18, No. 3, July 1984.
- [10] Glassner, Andrew "Space Subdivision for Fast Ray Tracing" IEEE Computer Graphics and Applications, October 1984.
- [11] Hockney, R. V. and Jesshope, C. R. "Parallel Computers: Architecture, Programming, and Algorithms" Adam Hilger Ltd. publisher, Bristol, 1981.
- [12] Kajiya, James T. "Ray Tracing Volume Densities" Computer Graphics (SIGGRAPH Proc.), Vol 18, No. 3, July 1984.
- [13] Kajiya, James T. "New Techniques for Ray Tracing Procedurally Defined Objects" ACM Transactions on Graphics, Vol 2, No. 3, July 1983.
- [14] Max, Nelson "Vectorized Procedural Models for Natural Terrain: Waves and Islands in the Sunset" Computer Graphics (SIGGRAPH Proc.), Vol. 15, No. 3, August 1981.

- [15] Max, Nelson "Computer Representation of Molecular Surfaces" IEEE Computer Graphics and Applications, Vol 3, No 5, pp 21-29, August 1983.
- [16] Mittelman, Phillip "Computer-Generated Images Bring Computer Power to Animation" Computer Technology Review, Summer 1984.
- [17] Palazzi, Maria "Ray Tracing and its Applications" Proceedings of Computer FX 84 Wembley, Online Publishers, Pinner UK 1984.
- [18] Roth, S. D. "Ray Casting for Modeling Solids" Computer Graphics and Image Processing, Vol 18, 1982.
- [19] Weghorst, H., Hooper, G. and Greenberg, D. "Improved Computational Methods for Ray Tracing" ACM Transactions on Graphics, Vol 3, No. 1, January 1984.
- [20] Weinberg, Richard "Parallel Processing Image Synthesis and Anti-Aliasing" Computer Graphics (SIGGRAPH Proc.), Vol. 15, No. 3, August 1981.
- [21] Whitted, Turner "Processing Requirements for Hidden Surface Elimination and Realistic Shading" Digest of Papers, COMPCON Spring '82, IEEE 1982.
- [22] Whitted, Turner "An Improved Illumination Model for Shaded Display" Communications of the ACM, Vol. 23, No. 6, June 1980.
- [23] Whitted, Turner "Image Synthesis Tutorial Notes" Eurographics 1984, Copenhagen.
- [24] "QUICKRAY: Non-Normal Incidence Three dimensional Ray Tracing Software" Sierra Geophysics, Seattle Washington 1984.
- [25] Cleary, John G., Brian Wyvill, Graham Birtwistle, Reddy Vatti, "Multiprocessor Ray Tracing" SIGGRAPH '85 Tutorial Notes.
- [26] Plunkett, David J., and Micheal J. Bailey, "The Vectorization of a Ray Tracing Algorithm for Improved Execution Speed" IEEE CGA, August 1985, pp. 52-60.
- [27] Scott, Ken, John Gustafson, and Stuart Hawkinson "Ray Tracing with the FPS Tesseract" unpublished, 1986.
- [28] Scott, Ken "Scientific Computers and Computer Graphics" CHECKPOINT, Vol 4, No. 5, May 1986. Floating Point Systems, Publishers.



Example 1. Ray Traced Cosmic Dust Particle.



Example 2. Fractal Terrain Surface Plot.